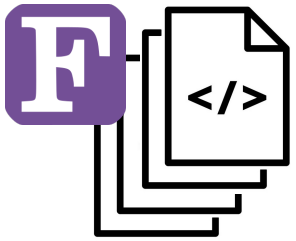
Yakup Koray Budanaz, Alexandru Calotoiu

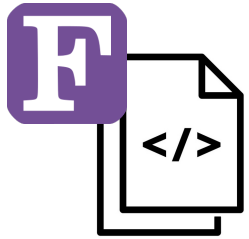
SoftHier End-of-the-Year-Two Meeting



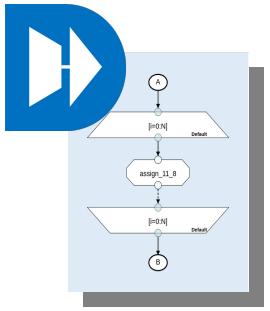
Absolute Performance For Weather Stencils



Original Fortran Sources



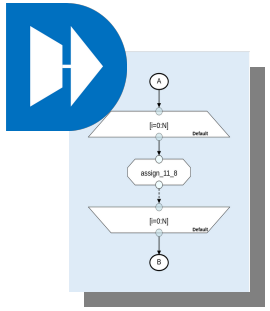
Preprocessed Fortran Sources (modules and functions inlined, macros expanded)



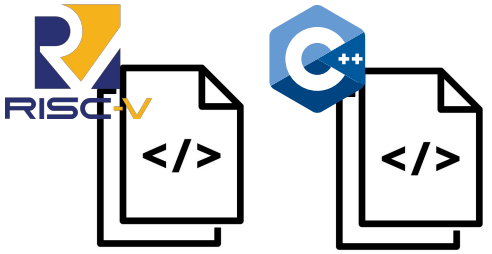
Unoptimized DaCe SDFGs

Can you directly apply scheduling transformations on a new accelerator? Just run -

```
.offload() ?
.vectorize ?
```

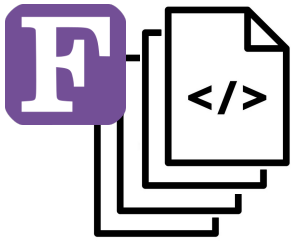


Optimized SDFG

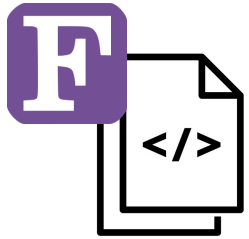


Generated Source Code

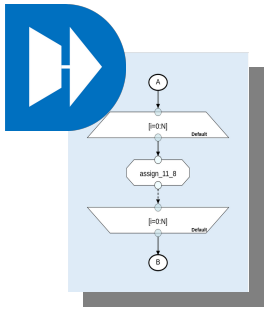
Absolute Performance For Weather Stencils



Original Fortran Sources



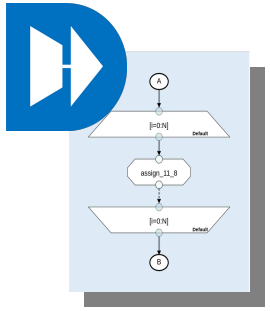
Preprocessed Fortran Sources (modules and functions inlined, macros expanded)



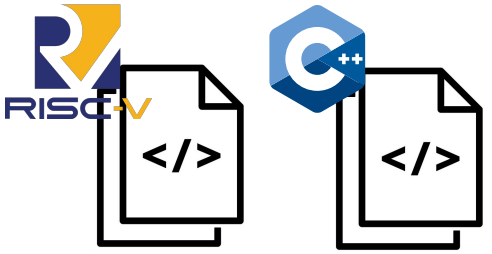
Unoptimized DaCe SDFGs

No.

new accelerator?
Just run -
`.offload() ?`
`.vectorize ?`

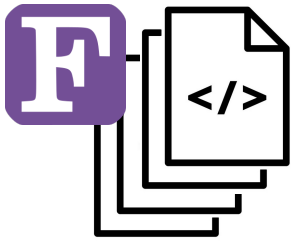


Optimized SDFG

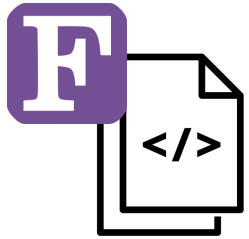


Generated Source Code

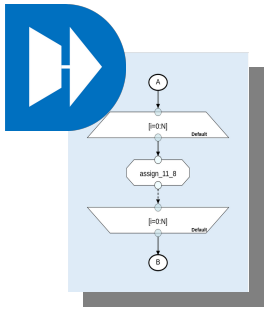
Absolute Performance For Weather Stencils



Original Fortran Sources



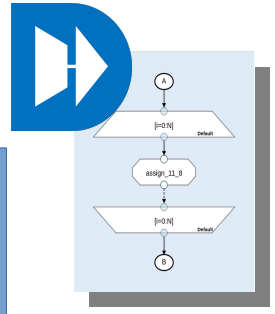
Preprocessed Fortran Sources (modules and functions inlined, macros expanded)



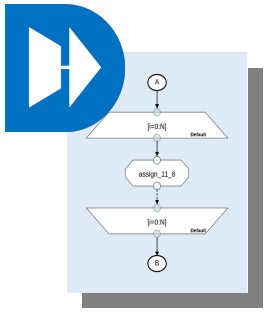
Unoptimized DaCe SDFGs

Need to first prepare the SDFG for the Accelerator

And then apply accelerator-specific optimizations

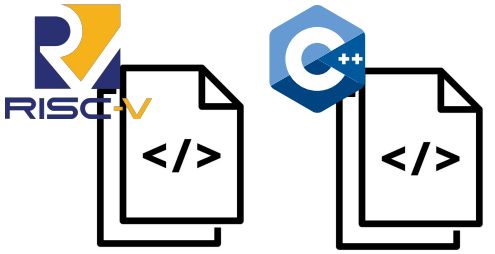


SDFG Amenable to Accelerator Optimization



Optimized SDFG (for the target accelerator)

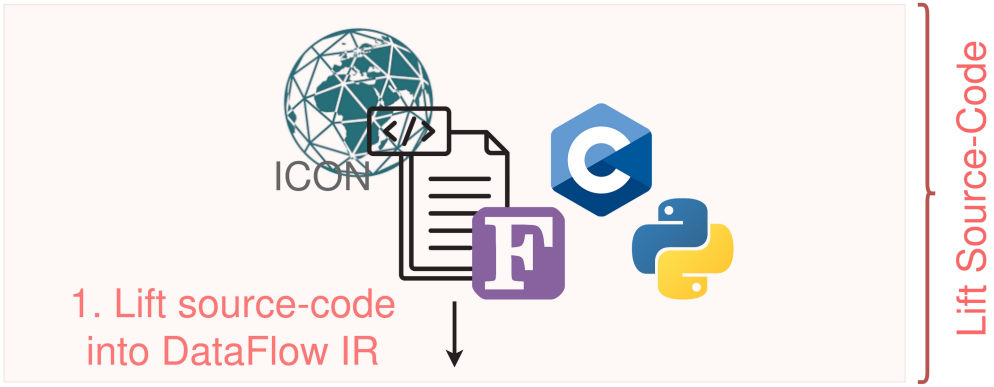
With this approach we obtain ~1.5x speed-up over expert-tuned code



Generated Source Code

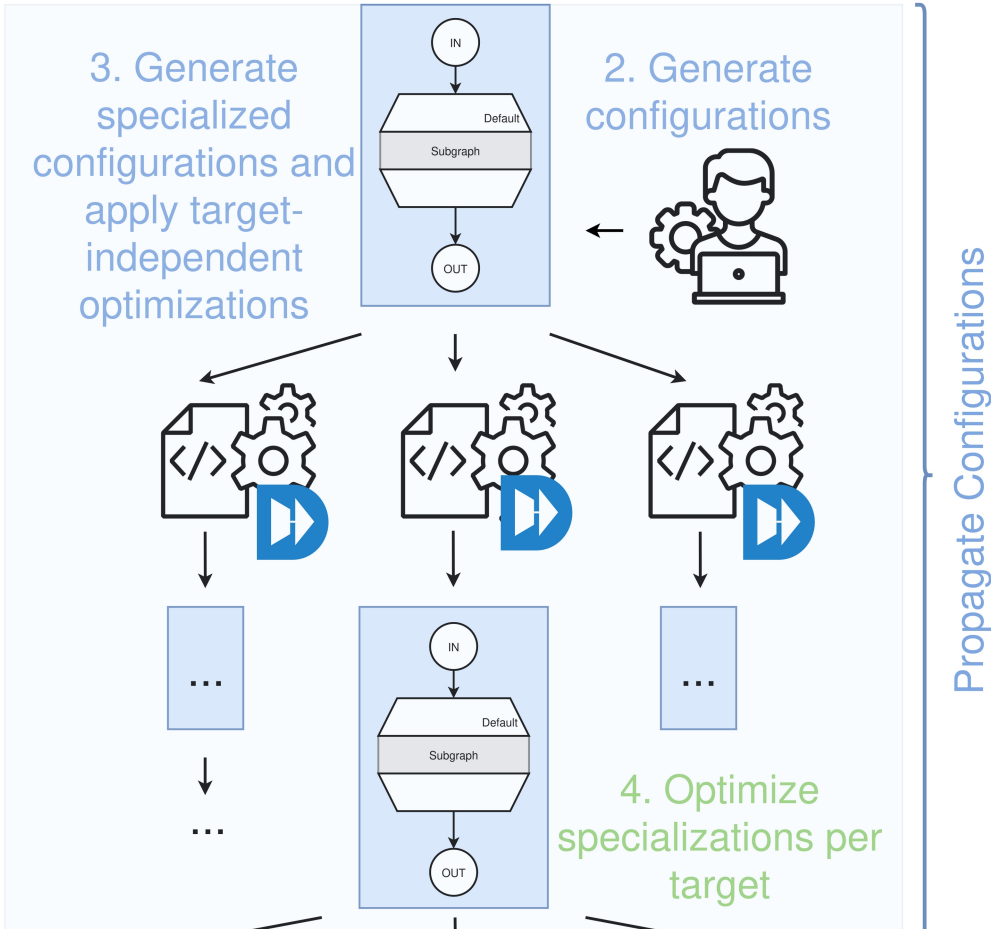
Optimizing ICON

- First step to optimize ICON involves lifting ICON's Fortran source code into the Stateful Dataflow MultiGraphs (SDFG) IR.



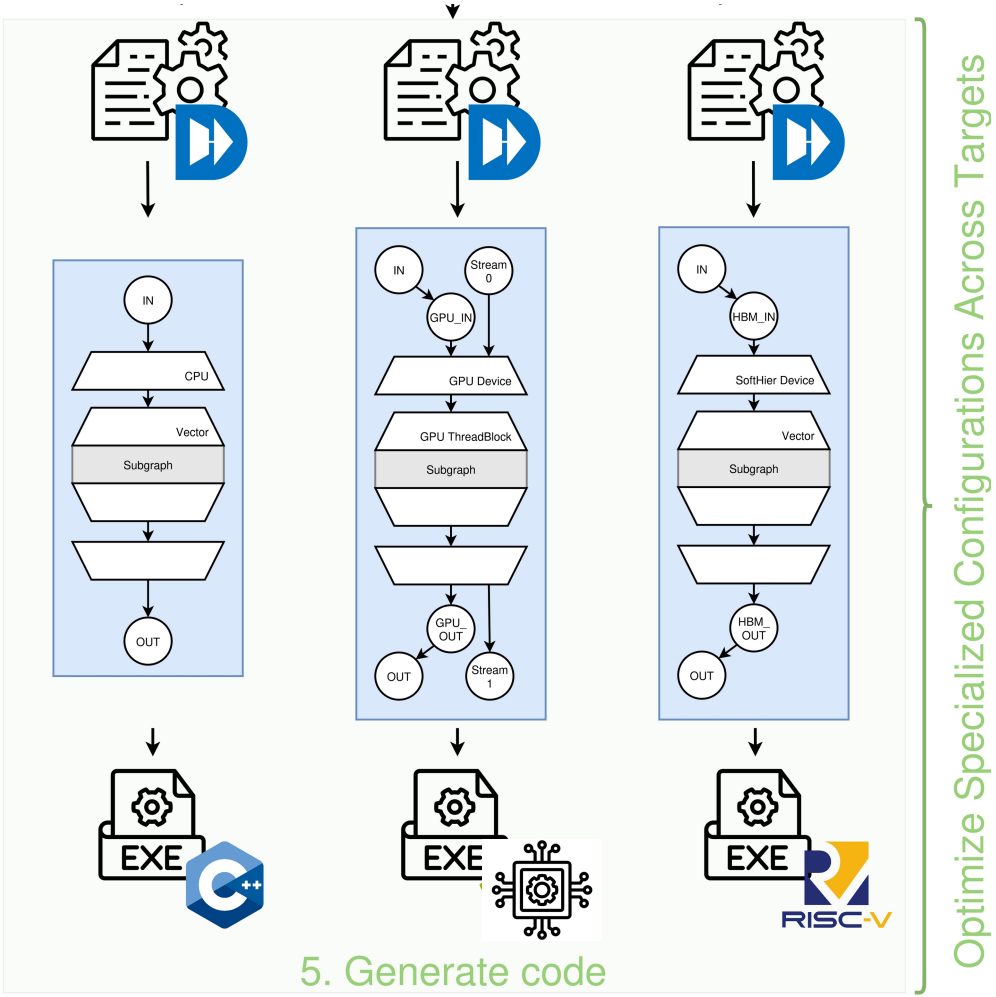
Optimizing ICON

- In second step we collect the configuration-dependent constant and possible configurations necessary by the configurations used for the Gordon-Bell run and obtain a specialized SDFG per configuration.



Optimizing ICON

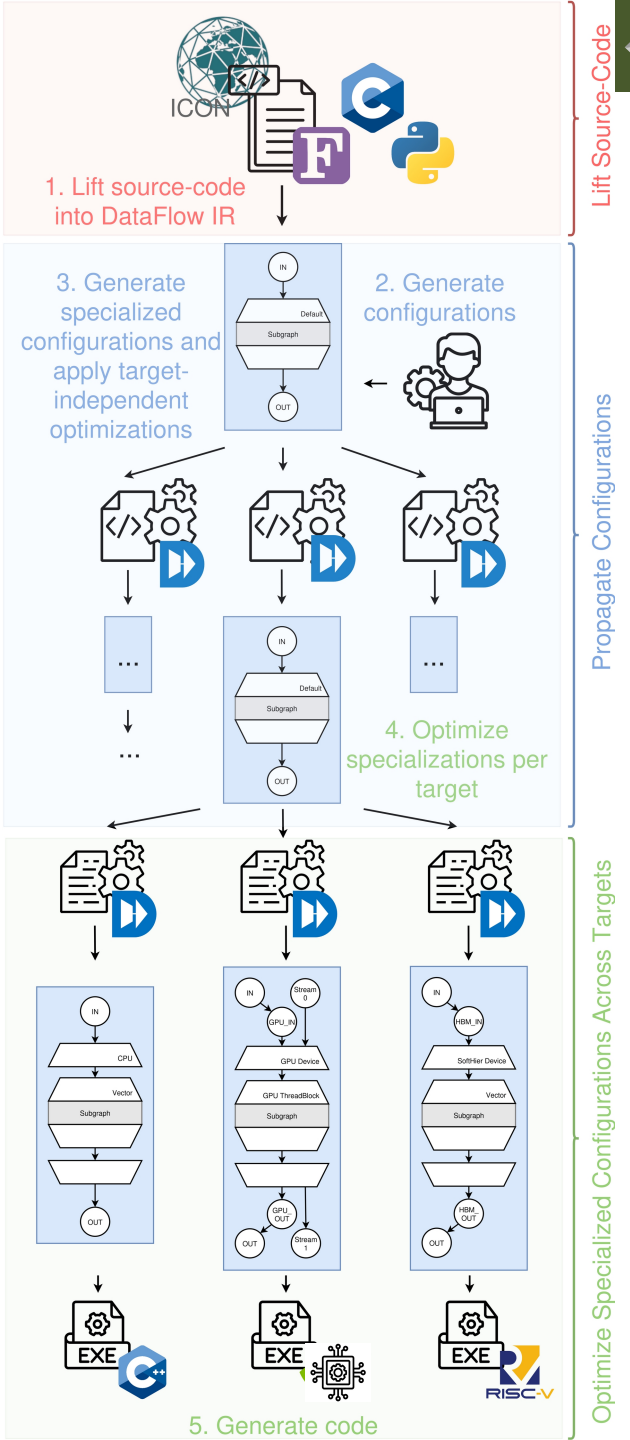
- We optimize the specialized SDFGs depending on the target hardware



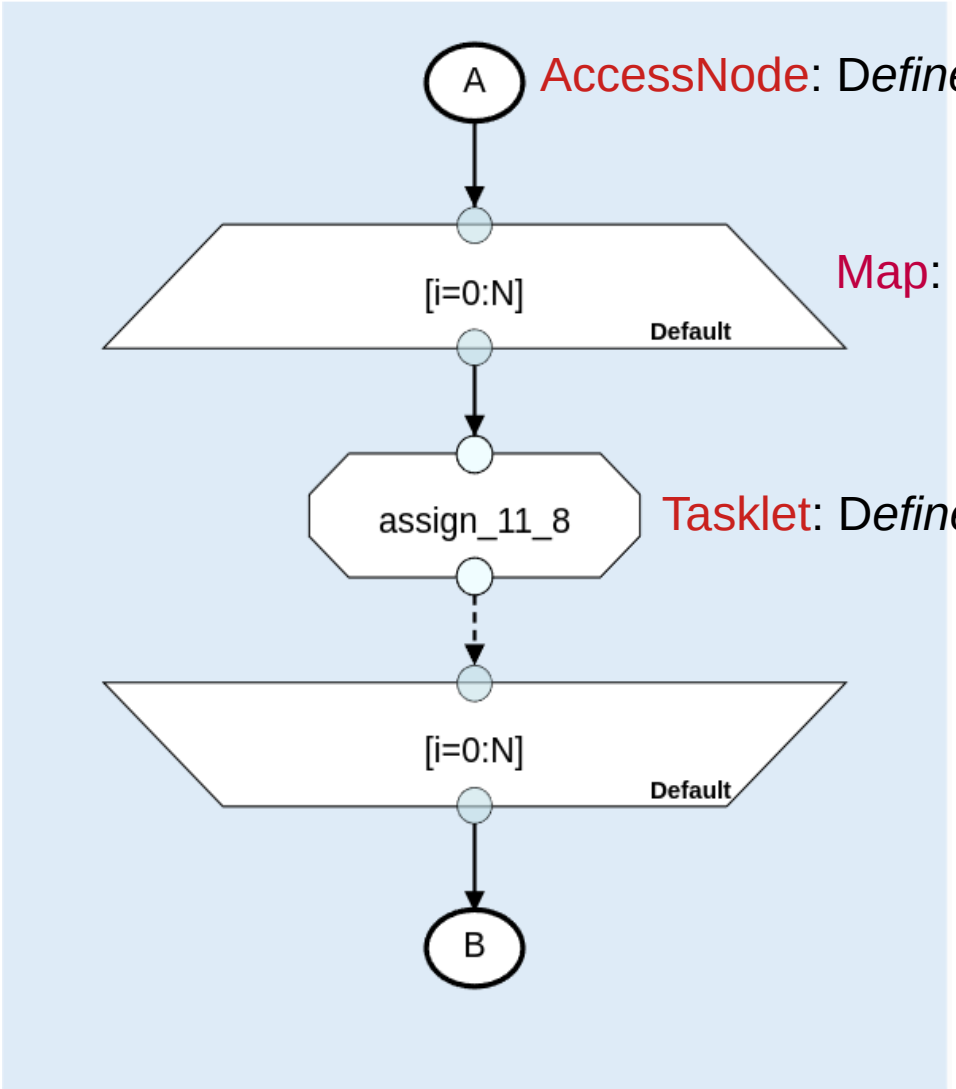
Optimizing ICON

- The full optimization pipeline looks as follows:
  - Lift control-flow centric IR into data-flow IR
  - Automatically or manually collect configurations and generate specialized SDFGs.
  - Optimize each specialized SDFG individually.

- → This automated pipeline also improves maintainability, while improving performance, by removing all pre-processor pragmas and decreasing LoC in Velocity Tendencies module of ICON by ~35%



How do we Decouple Optimizations from Source Code?
A Brief Introduction to the SDFG IR

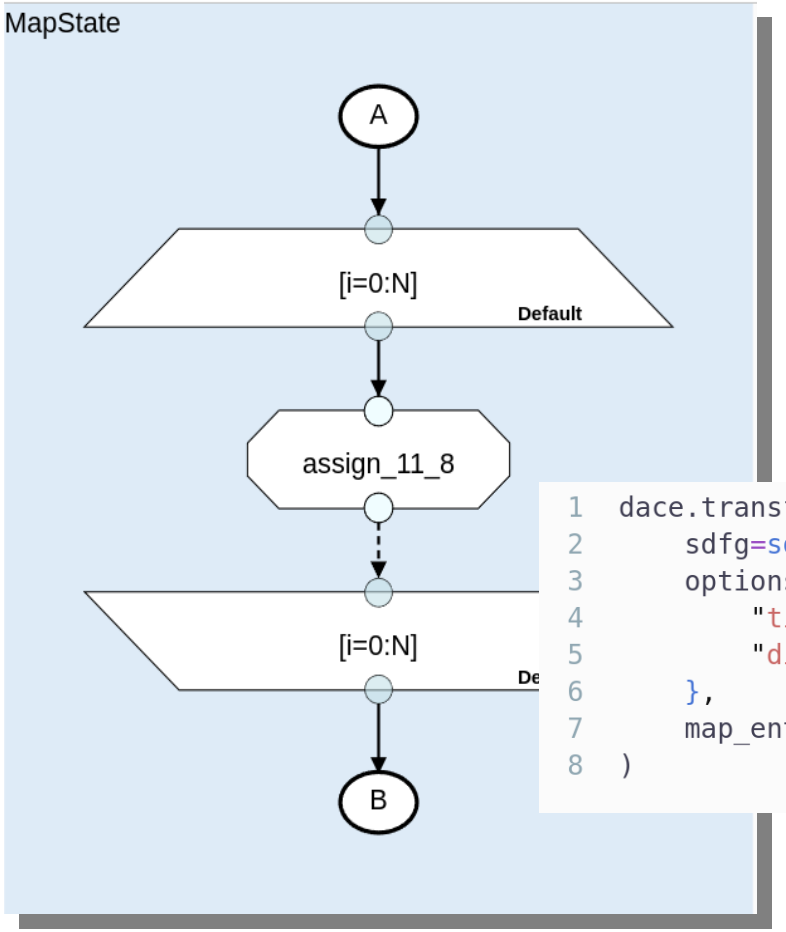


AccessNode: *Defines an access to a data container*

Map: *Defines a parallel region*

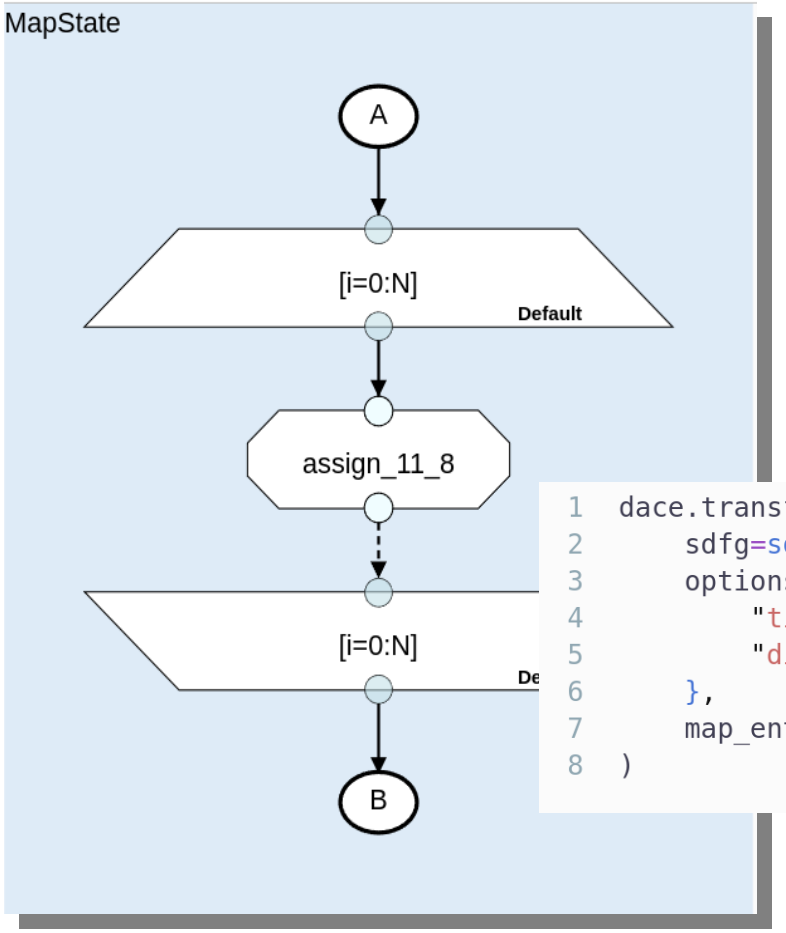
Tasklet: *Defines an operation on data*

How do we Decouple Optimizations from Source Code?
A Brief Introduction to the SDFG IR



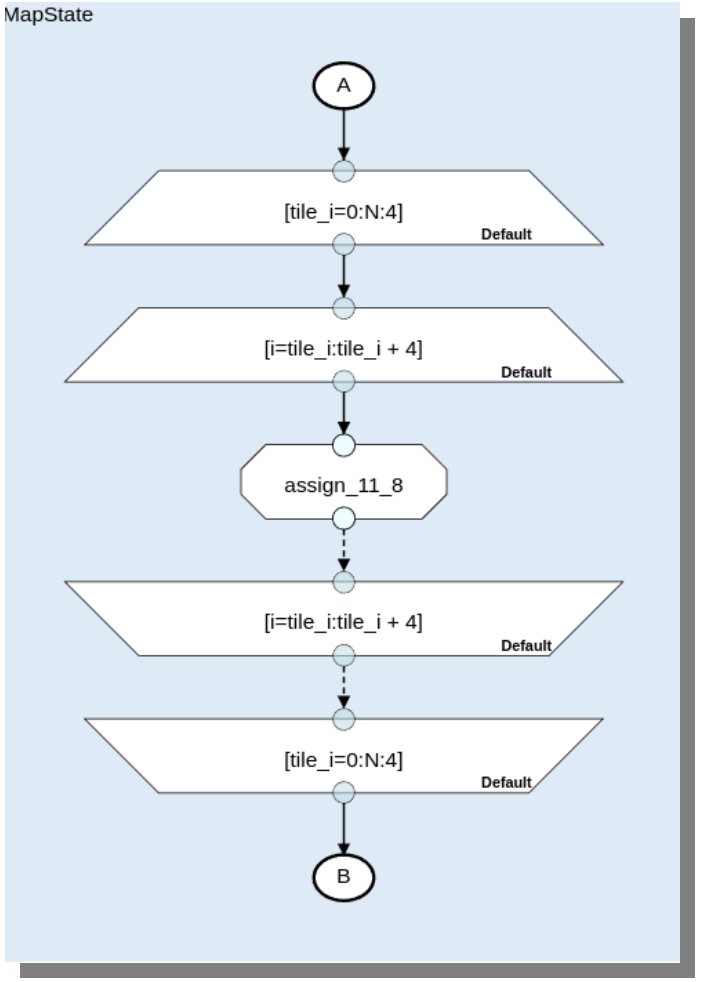

```
dace.transformation.dataflow.MapTiling.apply_to(
    sdfg=sdfg,
    options={
        "tile_sizes":(4,),
        "divides_evenly":True
    },
    map_entry=map_entry
)
```

How do we Decouple Optimizations from Source Code?
A Brief Introduction to the SDFG IR



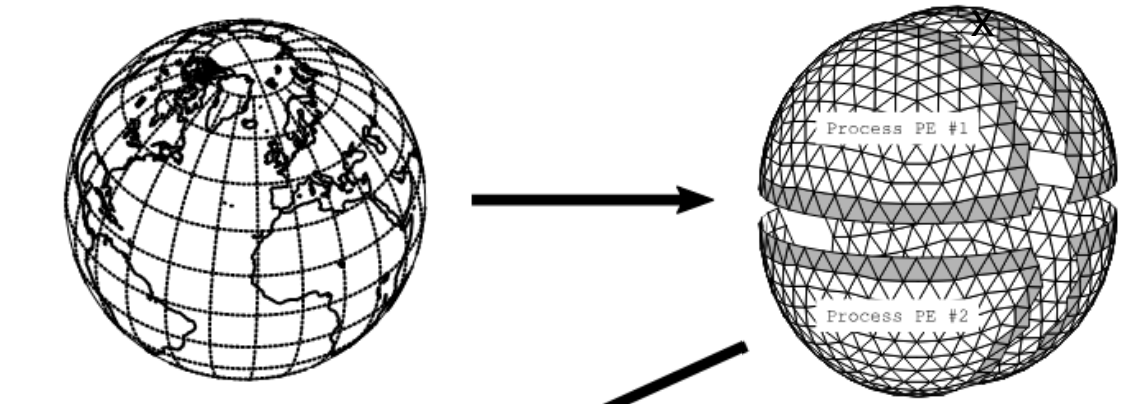
All operations are written as transformations on the IR

```
dace.transformation.dataflow.MapTiling.apply_to(
    sdfg=sdfg,
    options={
        "tile_sizes":(4,),
        "divides_evenly":True
    },
    map_entry=map_entry
)
```

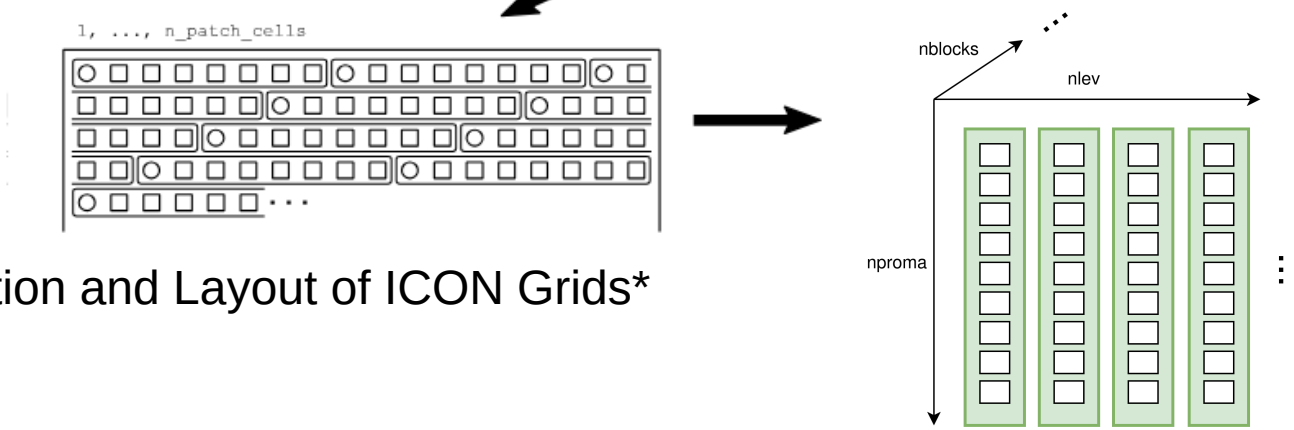


A Small Introduction to ICON's Weather Stencils

- Weather stencils are mostly semi-structured over two dimensions where the vertical dimension `nlev` is structured and `nproma * nbloks` domain is unstructured and blocked with the factor `nproma` for improved cache loaclity.




```
1  for i in 1..nbloks:
2      for k in 1..nlev:
3          for j in 1..nproma:
4              kernel_body_1()
5
```



Grid Decomposition and Layout of ICON Grids*

*https://www.dwd.de/DE/leistungen/nwv_icon_tutorial/nwv_icon_tutorial.html (ICON Tutorial 2024)

An Example for a Configuration Specific Optimization:

- We propagate all known configuration dependent constants and aggressively fuse kernels.

The Gordon-Bell configuration uses `trave_solver_type = 5`

```
1  for i in 1..nblocks:
2      for k in 1..nlev:
3          for j in 1..nproma:
4              kernel_body1()
5
6  if trace_solver_type == 5:
7      for i in 1..nblocks:
8          for k in 1..nlev:
9              for j in 1..nproma:
10                 kernel_body2()
```



```
1  for i in 1..nblocks:
2      for k in 1..nlev:
3          for j in 1..nproma:
4              kernel_body1()
5              kernel_body2()
```

An Example for a Configuration Specific Optimization:

- We propagate all known configuration dependent value sets and aggressively fuse kernels.
- For example `istep` can have the values `{1, 2}`

```
1  for i in 1..nblocks:
2      for k in 1..nlev:
3          for j in 1..nproma:
4              kernel_body3()
5
6  if istep == 1:
7      for i in 1..nblocks:
8          for k in 1..nlev:
9              for j in 1..nproma:
10                 kernel_body4()
11
12 if istep == 2:
13     for i in 1..nblocks:
14         for k in 1..nlev:
15             for j in 1..nproma:
16                 kernel_body5()
17
18 for i in 1..nblocks:
19     for k in 1..nlev:
20         for j in 1..nproma:
21             kernel_body6()
```

Specialize for `istep == 1`

```
1  # if istep == 1
2  for i in 1..nblocks:
3      for k in 1..nlev:
4          for j in 1..nproma:
5              kernel_body3()
6              kernel_body4()
7              kernel_body6()
```

Specialize for `istep == 2`

```
1  # if istep == 2
2  for i in 1..nblocks:
3      for k in 1..nlev:
4          for j in 1..nproma:
5              kernel_body3()
6              kernel_body5()
7              kernel_body6()
```

An Example For A Target-Specific Optimization:

- We perform optimizations to increase the kernel dimension sizes even if it means we duplicate computation.

```
1  for i in 1..nbloks:
2      for k in 1..nlev:
3          if condition_1(k):
4              for j in 1..nproma:
5                  kernel_body_1()
```

Schedule Kernels →

```
1  for i in 1..nbloks: # Host-side Loop
2      for k in 1..nlev:  # Acc. Kernel
3          if condition_1(k):
4              for j in 1..nproma:
5                  kernel_body_1()
6
```

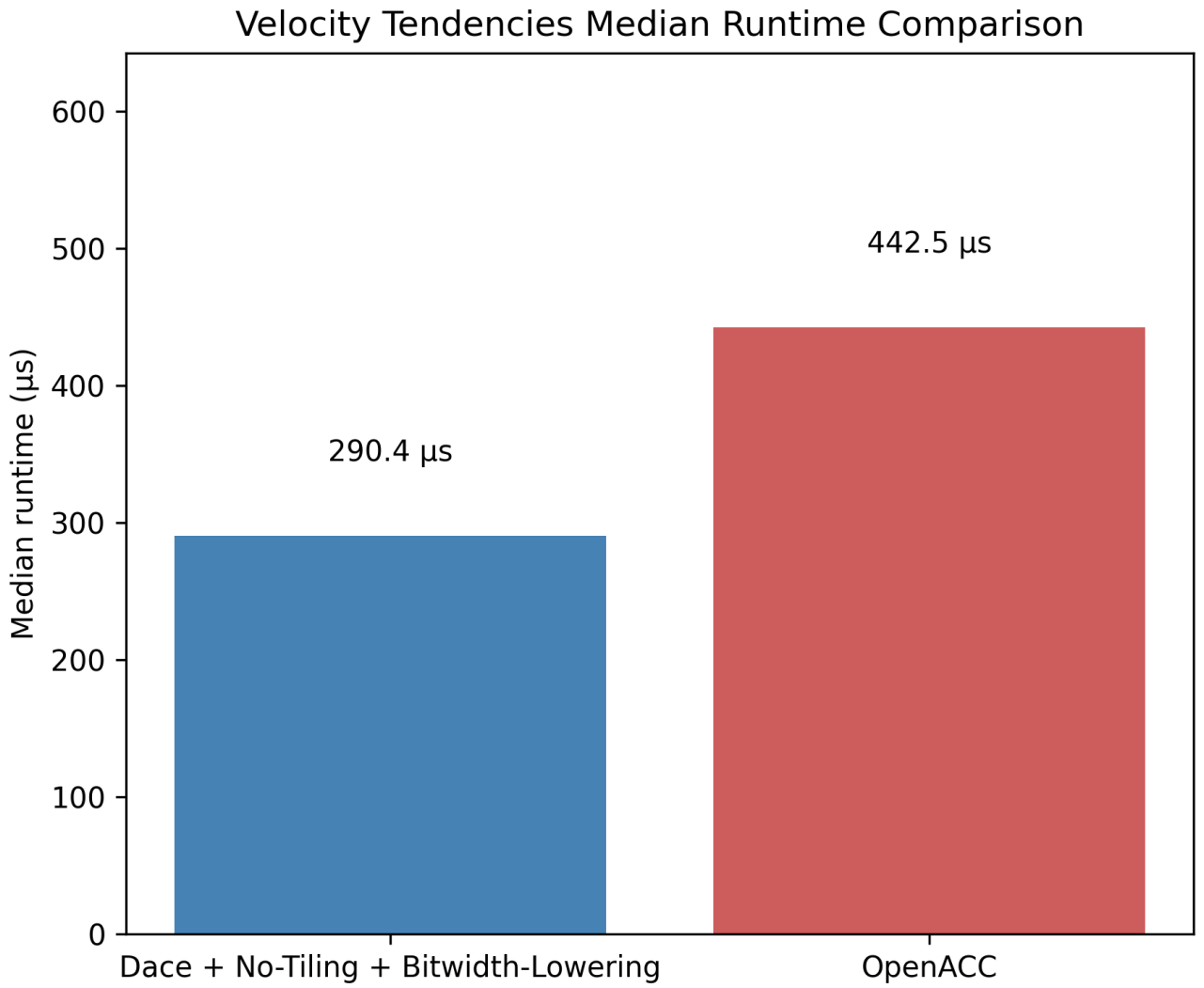
Move If Inside Loop
(to extend kernel dimensions)

```
1  for i in 1..nbloks:
2      for k in 1..nlev:
3          for j in 1..nproma:
4              if condition_1(k):
5                  kernel_body_1()
6
```

Schedule Kernels →

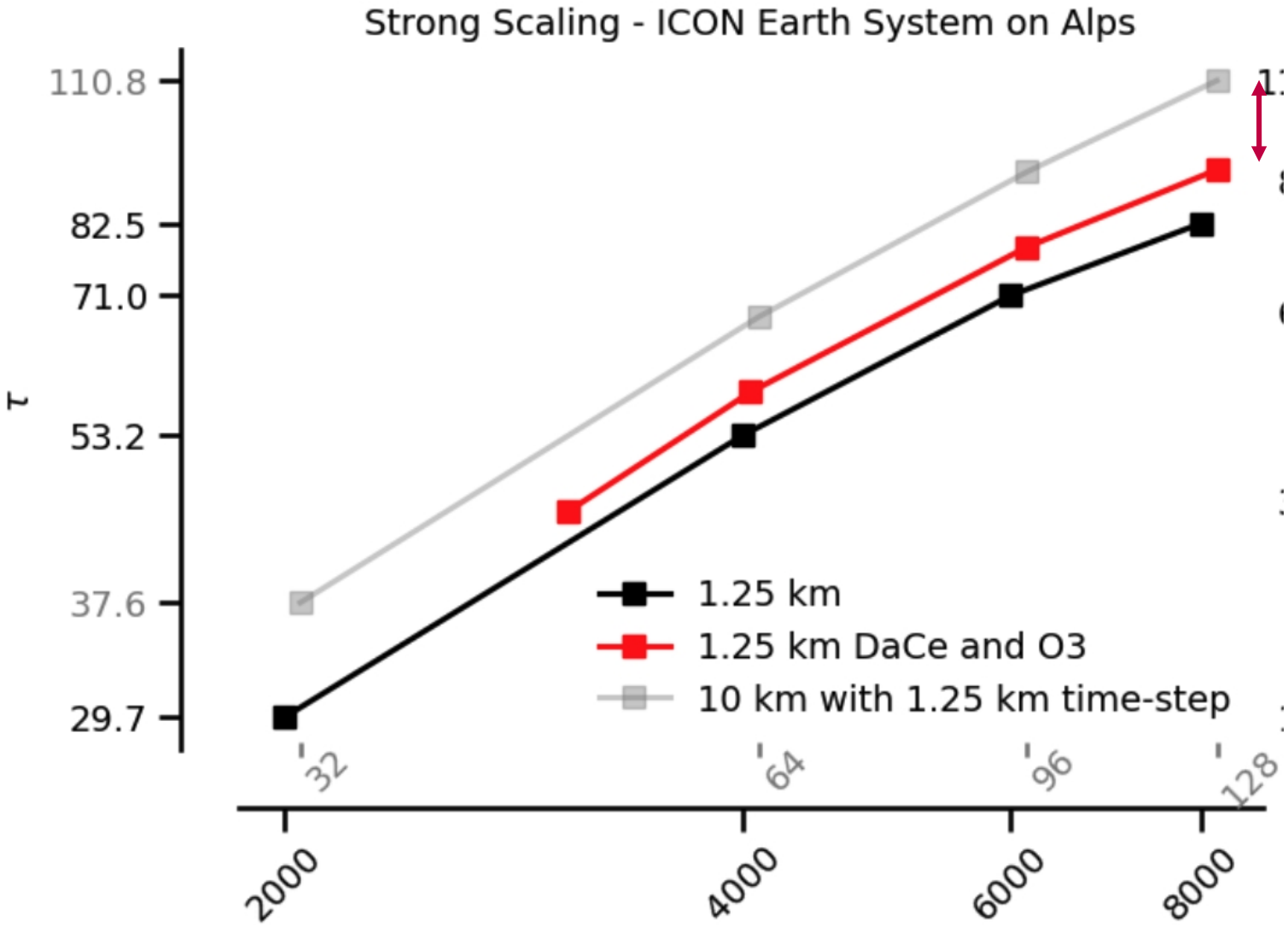
```
1  for i in 1..nbloks: # Host-side Loop
2      # Acc. Kernel
3      for k, j in cross(1..nlev, 1..nproma):
4          if condition_1(k):
5              kernel_body_1()
```

Impact of the Optimizations:



1.5x speedup by applying configuration-propagation

Impact of the Optimizations:

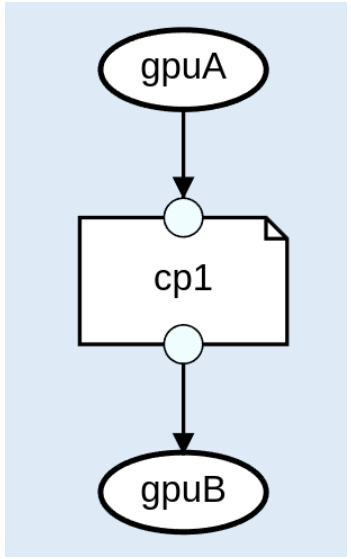


1.2x speedup on optimized section of ICON.

1.1x overall over expert-tuned OpenACC code, over complete ICON

Specialization of Reductions, Copies and Memsets

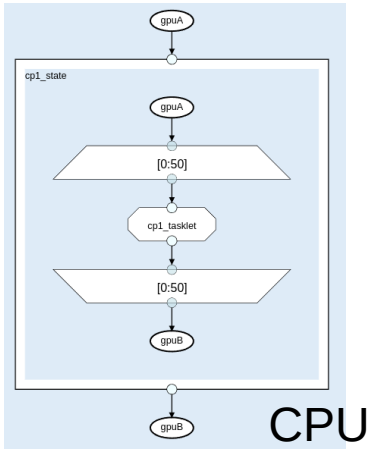
- Existing optimizations strategies rarely work out-of-the-box for new accelerators. Thus we specialize the reductions depending on the target.



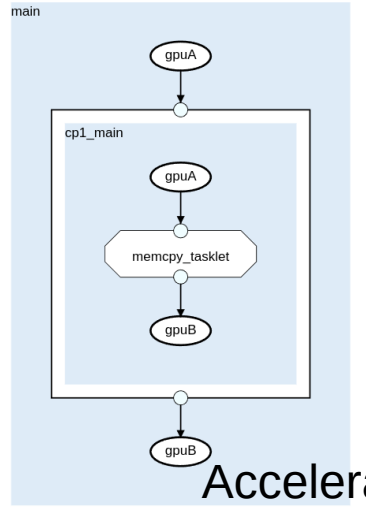

```
.library.specialize(
        Pure
)
```

```
.library.specialize(
        Accelerator
)
```

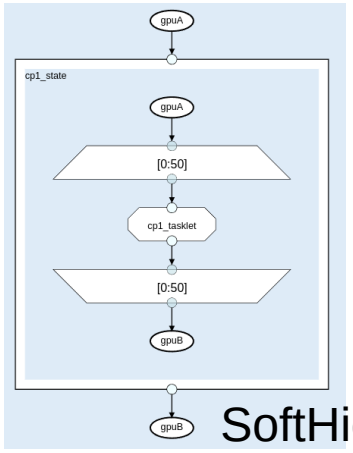
```
.library.specialize(
        SoftHier
)
```



CPU



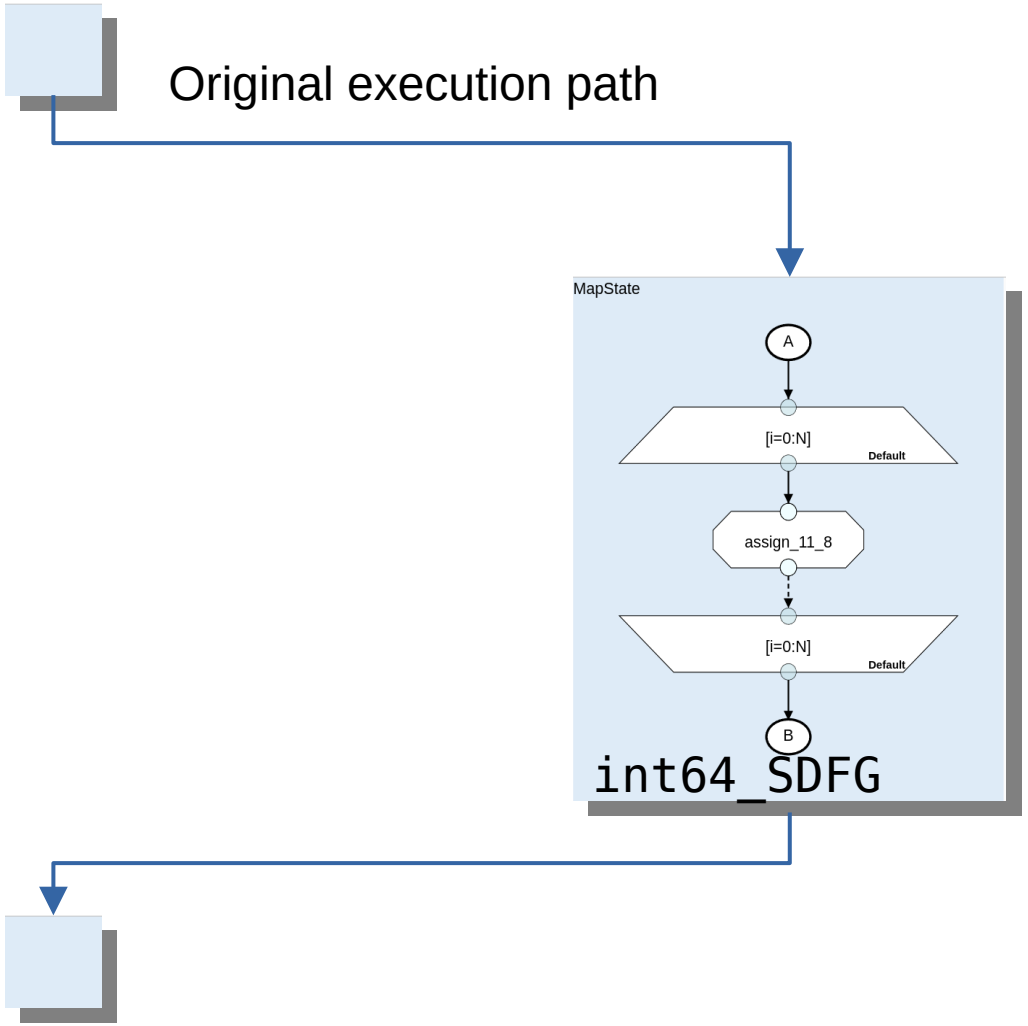
Accelerator



SoftHier

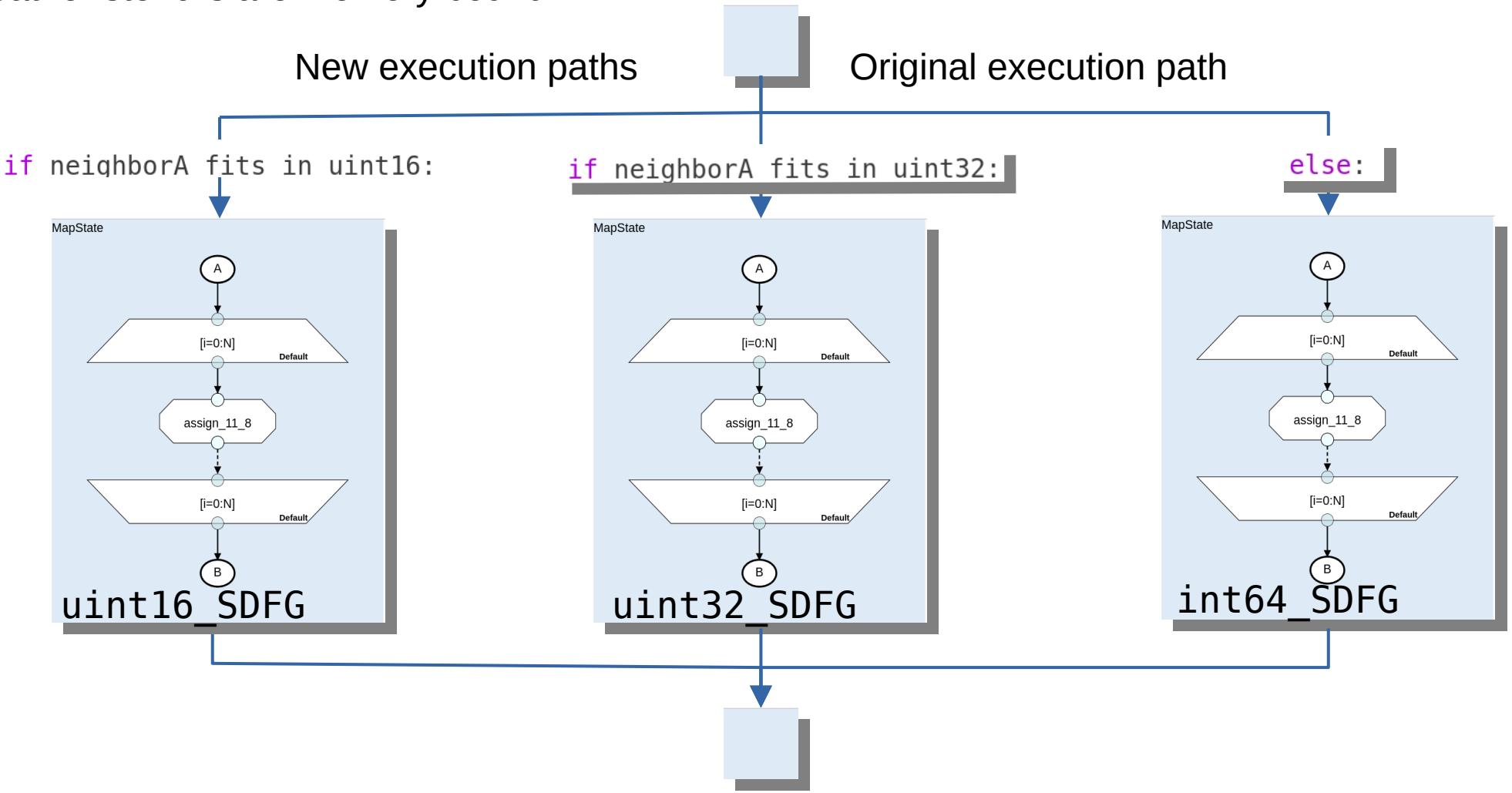
Specialization of NeighborLists

- Making all computation in FP64 and declaring everything in `int64` is not always necessary
- And weather stencils are memory bound!



Specialization of NeighborLists

- Making all computation in FP64 and declaring everything in `int64` is not always necessary
- And weather stencils are memory bound!



Gordon Bell

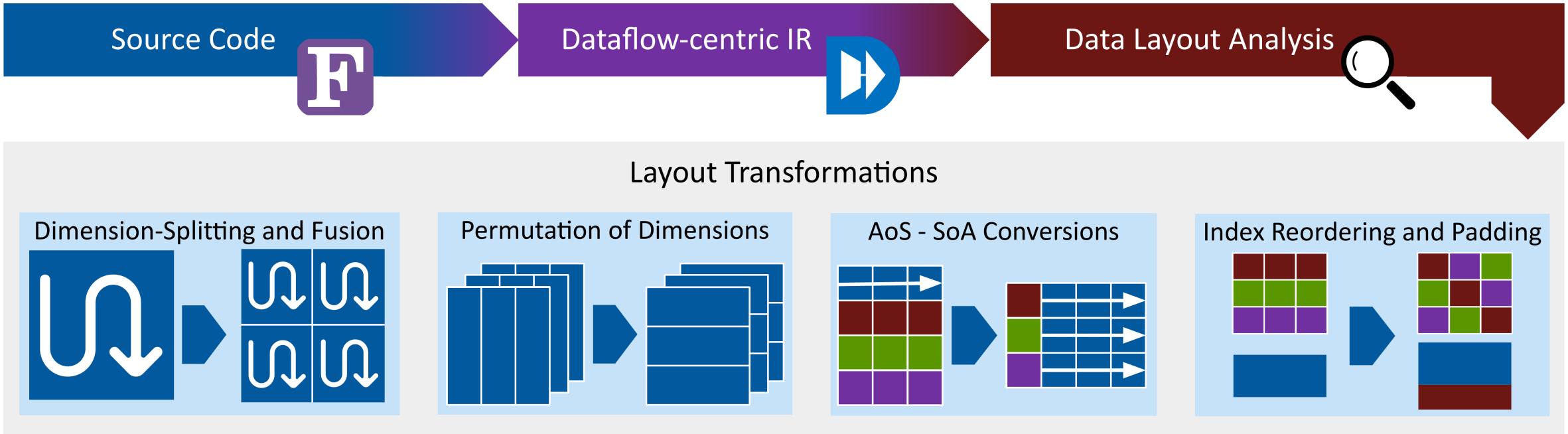
- Extra slide if we get the GB award

SIMT vs SIMD

- The Gordon-Bell run was performed on SIMT-model machines
- We have worked on completing the transition from SIMT-model machine to a SIMD-only model and a machine.

Layout Transformations

- We also optimized layouts of transient data to improve access coalescing and enable transformations from the SIMT model to SIMD model.
- Layout transformations and branch-elimination is necessary to enable a complete transition from SIMT to SIMD model.



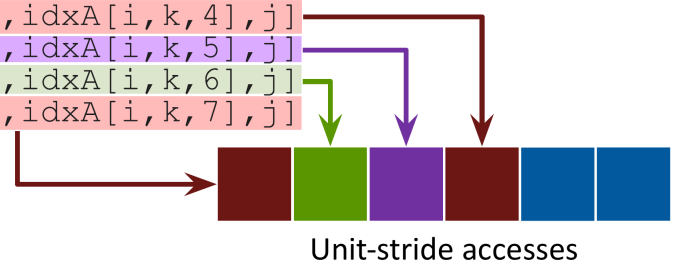
A Motivating Example for Layout Transformation from Weather Stencils

- Layout transformation enable the use of SIMD instructions for semi-structured weather stencils

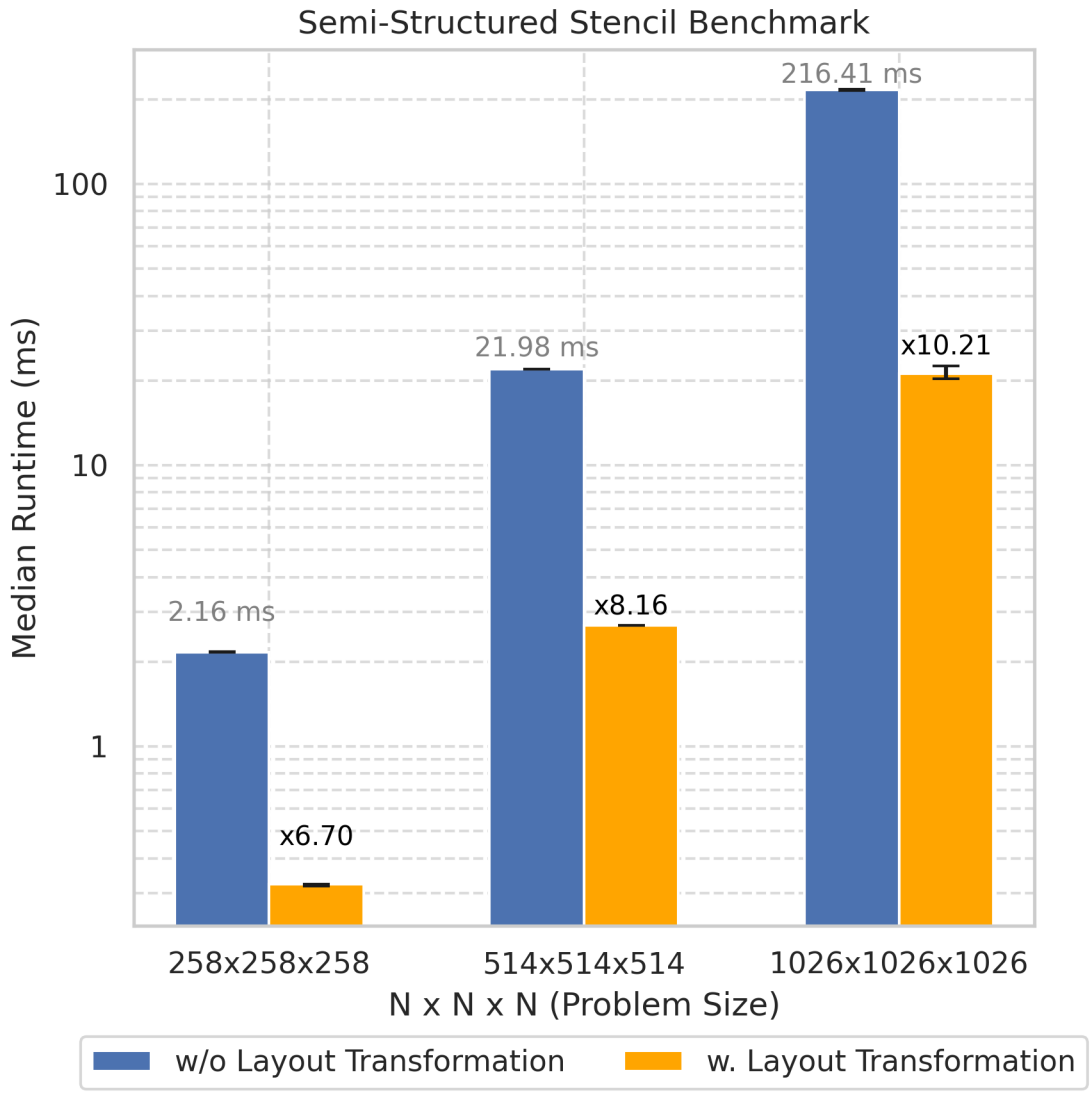
```
for i in [1:N-1]:
  for j in [1:N-1]:
    for k in [1:N-1]:
      B[i,j,k] = c * (A[i,j,k] + A[i,j-1,k] + A[i,j+1,k]
        + A[idxA[i,k,0],j,idxA[i,k,4]]
        + A[idxA[i,k,1],j,idxA[i,k,5]]
        + A[idxA[i,k,2],j,idxA[i,k,6]]
        + A[idxA[i,k,3],j,idxA[i,k,7]]
      )
```




```
for i in [1:N-1]:
  for k in [1:N-1]:
    for j in [1:N-1]:
      B[i,k,j] = c * (A[i,k,j] + A[i,k,j-1] + A[i,k,j+1]
        + A[idxA[i,k,0],idxA[i,k,4],j]
        + A[idxA[i,k,1],idxA[i,k,5],j]
        + A[idxA[i,k,2],idxA[i,k,6],j]
        + A[idxA[i,k,3],idxA[i,k,7],j]
      )
```

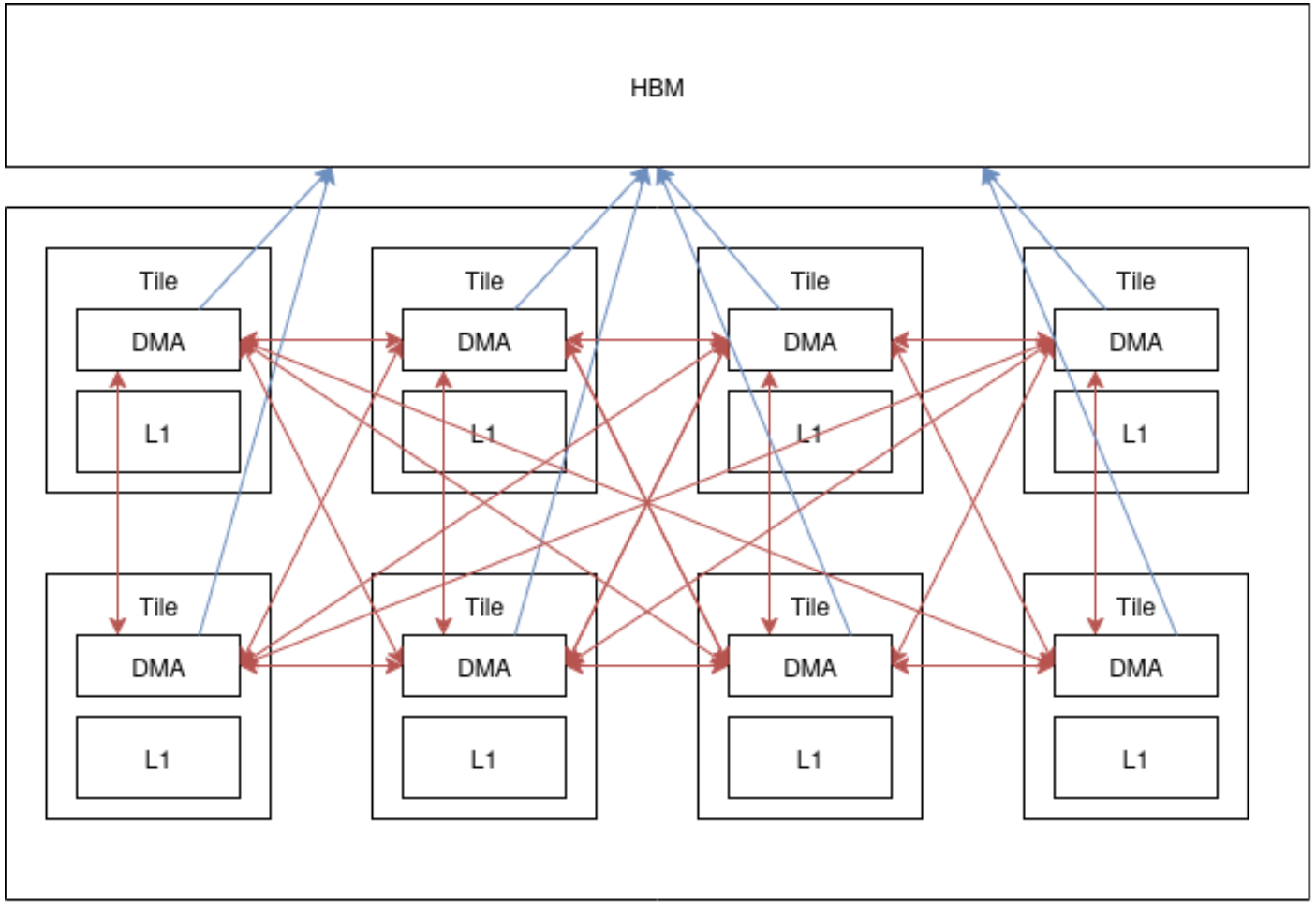


A Motivating Example for Layout Transformations from Weather Stencils



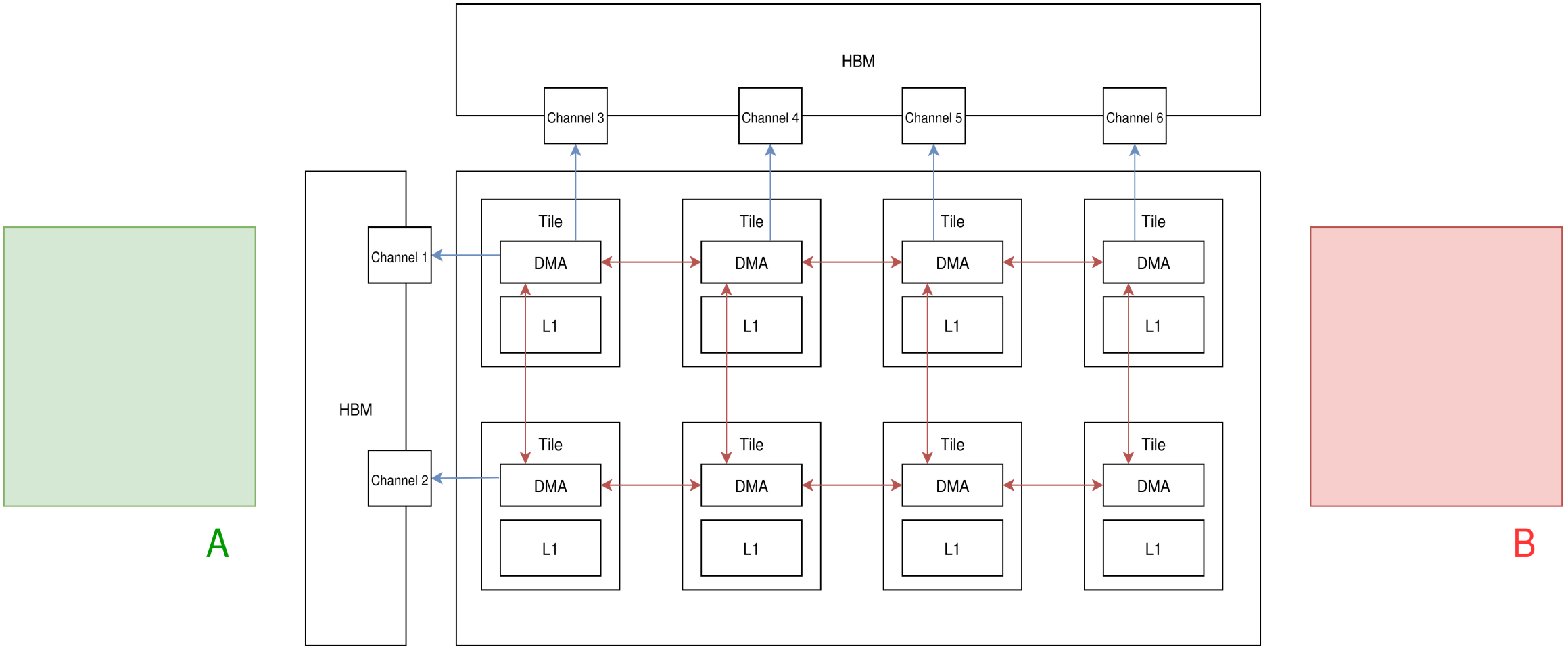
Layout Transformations On SoftHier

- The layout transformations implemented in DaCe are also applicable to the SoftHier simulator, where it provides an all-to-all abstraction for both accesses over the NoC and the HBM



Layout Transformations On SoftHier

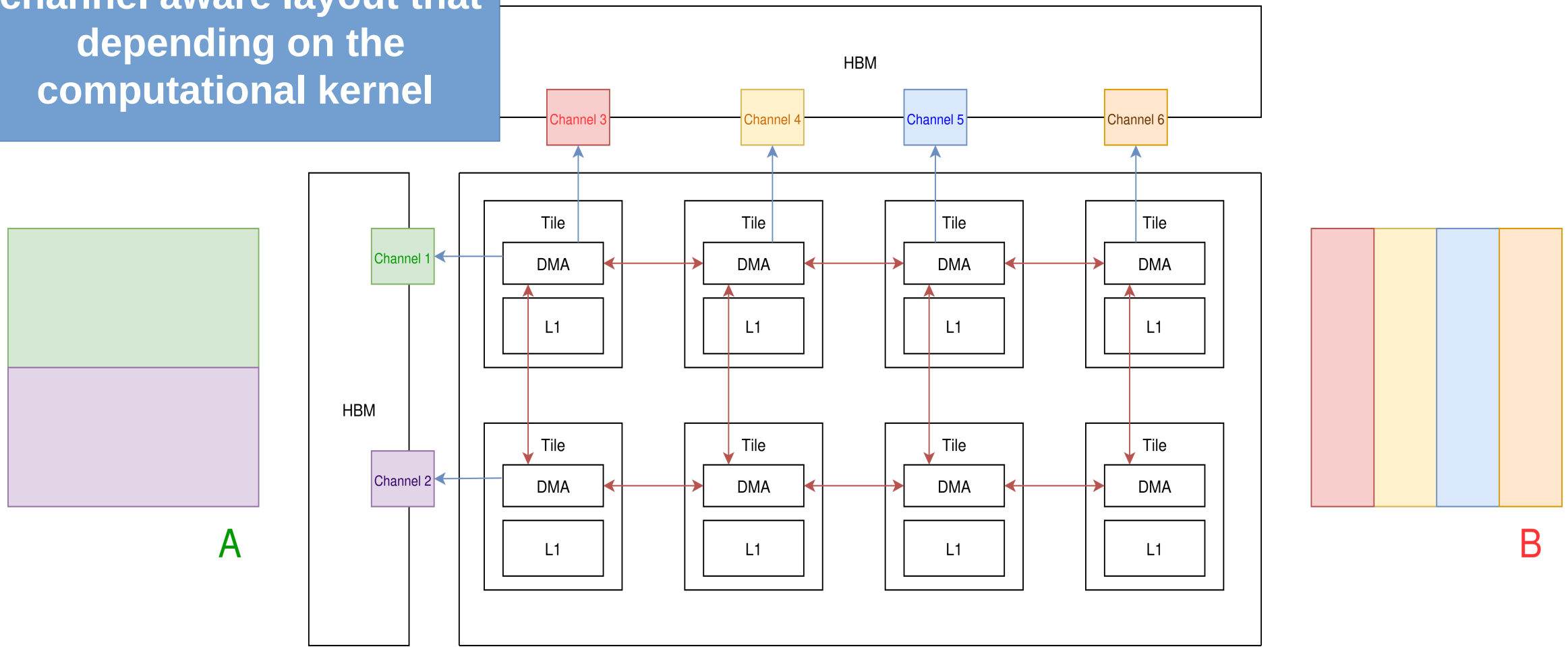
- We have helped in applying layout transformations for the SoftHier simulator



Layout Transformations On SoftHier

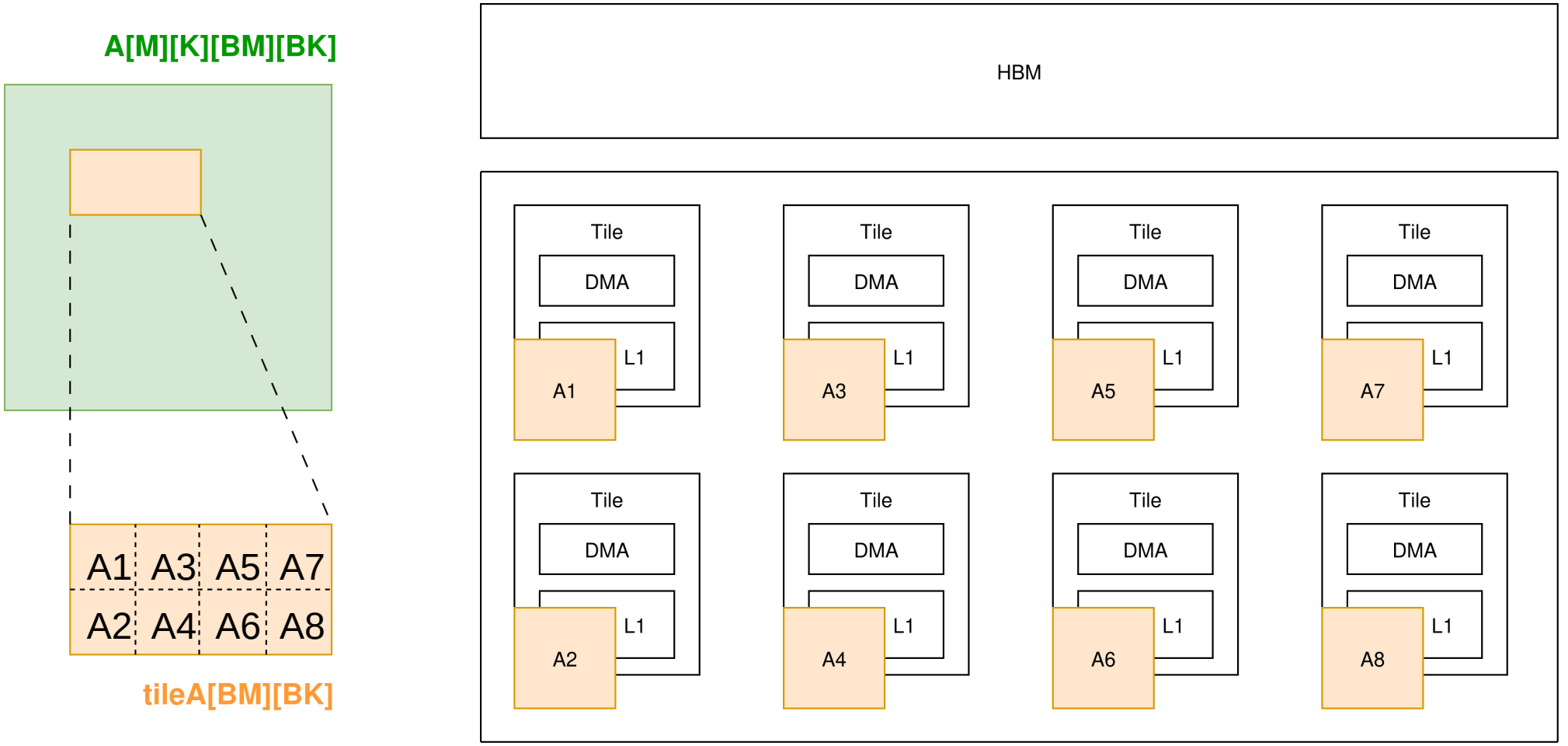
We identify the HBM-channel aware layout that depending on the computational kernel

- We have helped in applying layout transformations for the SoftHier simulator



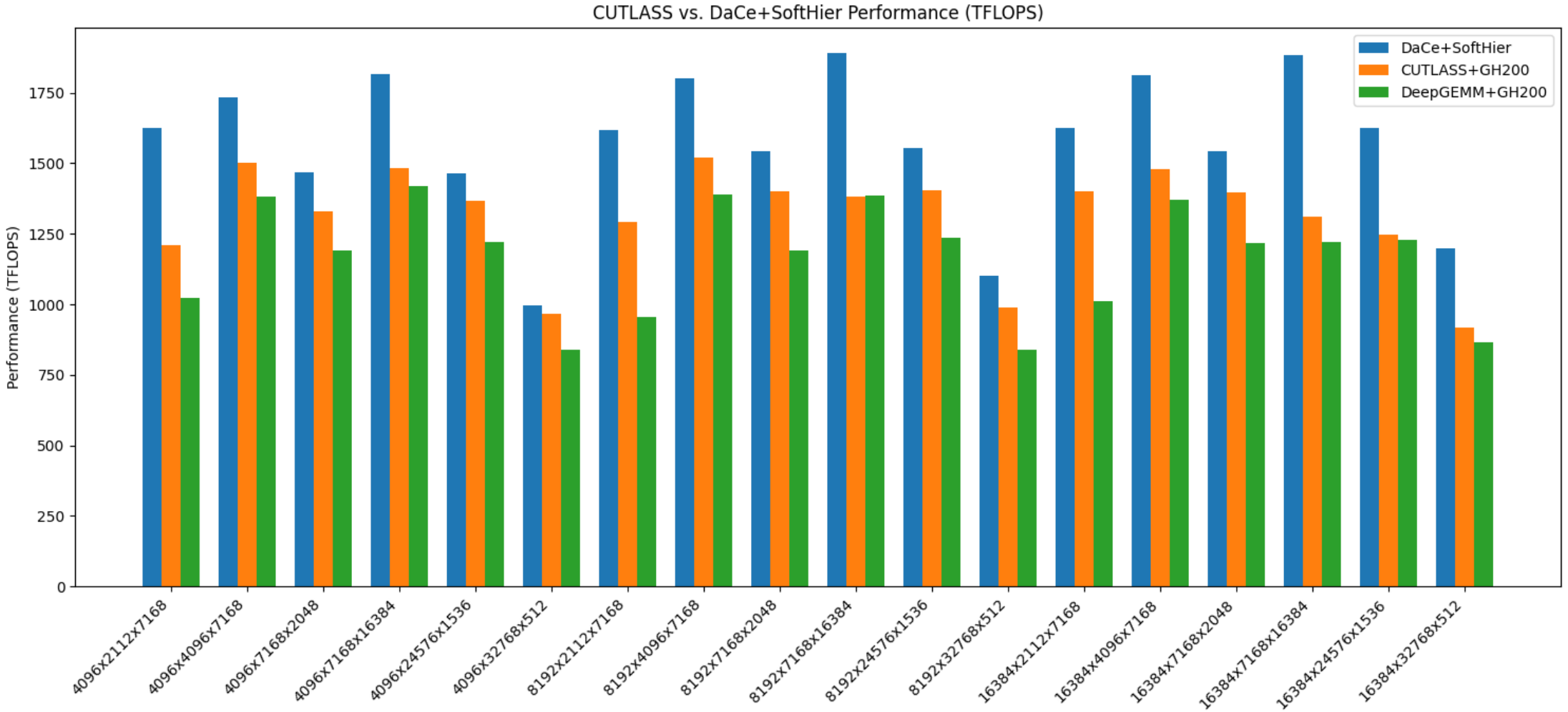
Layout Transformations On SoftHier

- We have helped in applying layout transformations for the SoftHier simulator



Layout Transformations On SoftHier

- Results obtained by a student supervised by Chi and Yakup



SoftHier on General Vectors: Auto-Vectorization

- Vector-Processors and Branches are Natural Enemies!

```
for i in range(N):
    if cond:
        A[i] = computation1(...)
    else:
        A[i] = computation2(...)
```



Eliminating branches is necessary for vectorization

```
for i in range(N):
    fcond = 1.0 if cond else 0.0
    A[i] = computation1(...) * fcond + \
           computation2(...) * (1.0 - fcond)
```

DaCe Vectorization

- DataFlow IR and having the global-view of the program enables vectorization of patterns not vectorizable by traditional compilers such as LLVM:

```
def f1(i):
    A[i] = A[i + B[i]] *C[i]
    A[i] = A[i] * D[i]
    # Further expressions

for i in range(N):
    f1(i)
```

Complete f1 can't be vectorized by LLVM due to the indirect access!

We can vectorize the function f1 partially.

```
def f1(i):
    A_packed = pack_load(A, i + B[i])
    A[i] = A_packed * C[i]
    A[i] = A[i] * D[i]
    # Further expressions

for i in range(0, N, 8):
    // Automated inlining
    A_packed[0..8] = pack_load(A, i..(i+8) + B[i..(i+8)])
    A[i..(i+8)] = A_packed[0..8] * C[i..(i+8)]
    A[i..(i+8)] = A[i..(i+8)] * D[i..(i+8)]
    # Further expressions
```

DaCe Vectorization

- Performance results TODO